# SCEM Final Coursework

Task II

CIRENYANGZHEN (2679375)

---

## IMPORTANT NOTES:

- **DO NOT** change the code block names. Enter your solutions to each question into the predefined code blocks.
- **Don't forget to replace the placeholder value in code block "Setup" by your valid student ID number.**
- If you want to use any packages, add their names to the vector `required.packages` in the code block "Setup". That code block will automatically install those packages (if needed) and load them. Any other calls to `install.packages()`, `library()` or `require()` may result in zero marks for the activity where it happens.
- Make sure that the data files *df.rds* and *df_holdout.rds* are located in the same folder as this file, *Task02.Rmd*.
- Instructions for this task are provided in blue text below. Your solution should NOT be added within those text boxes, and should NOT be in blue text. **During marking, all the text within the text blocks below, with names starting with "prompt", will be removed and not visible to the marker.**

```
##
## List of allowed packages (pre-loaded):
##   tidyverse
##   tidymodels
##   recipes
##   parsnip
##   bonsai
##   healthyR.ai
##   knitr
##   DataExplorer
##   themis
```

---

In this task you will build a full predictive pipeline for the problem described in the coursework specs. Please read each activity carefully and follow the instructions to complete your coursework.

Make sure that the data file **df.rds** is placed in the same folder as this Task02.Rmd file. The data that you can use for model development in this task will then be automatically loaded and set up for you, and stored into a dataframe variable called `df`.

---

## 1. Exploratory Data Analysis

**Note**: It is expected that this section should result in no more than 600-ish words (possibly less; not counting the code blocks).

Add here your exploratory data analysis. Your solution should include both your code (which markers must be able to run independently) and your rationale for each exploratory step. In general, your EDA steps should include: - A short justification of what a given plot or statistical summary is intended to reveal about the data.

- A code block implementing your EDA step.

At the end of your EDA section, you should also include a short commentary on what your data exploration revealed about the data, and which pre-processing steps may be required based on that.

You can use as many code blocks as needed, but don't overdo it (remember - everything needs to have a proper rationale).

### EDA CODE BLOCK 1-dataset structure analysis

In the first step, I summarized the basic structure of the dataset by checking the numbers of observations, variables, feature columns, and groups. Besides, I also checked variable types and the class distribution.

```
cat("number of observations:", nrow(df),"\n")
## number of observations: 7403
cat("number of variables", ncol(df), "\n")
## number of variables 390
cat("number of feature columns:", sum(grepl("^feat_", names(df))), "\n")
## number of feature columns: 385
cat("number of groups:", n_distinct(df$Info_group), "\n")
## number of groups: 550

var_types <- df %>%
  summarise(across(everything(), ~ class(.x)[1])) %>%
  pivot_longer(
    everything(),
    names_to  = "variable",
    values_to = "type"
  )

head(var_types)
## # A tibble: 6 x 2
##   variable             type
##   <chr>                <chr>
## 1 Info_PepID           character
## 2 Info_group           integer
## 3 Info_protein_id      character
## 4 Info_pos             numeric
## 5 feat_local_Entropy   numeric
## 6 feat_local_MolWeight numeric

var_types %>%
  count(type)
## # A tibble: 3 x 2
```

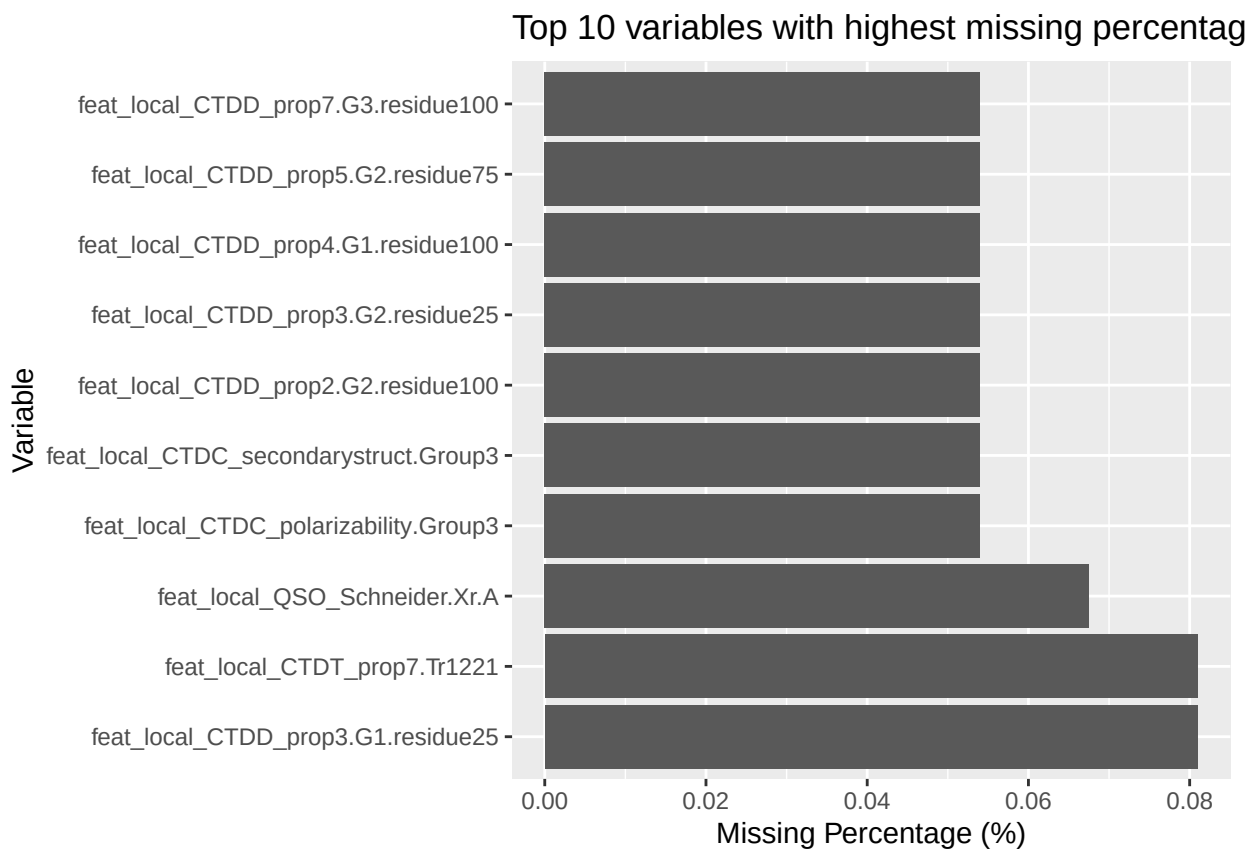
```
##   type          n
##   <chr>     <int>
## 1 character     2
## 2 integer       1
## 3 numeric     387

class_dist <- df %>%
  mutate(Class = as.factor(Class)) %>%
  count(Class) %>%
  mutate(percent = round(n/sum(n)*100, 1))
print(class_dist)
## # A tibble: 2 x 3
##   Class     n percent
##   <fct> <int>   <dbl>
## 1 -1     4342    58.7
## 2 1      3061    41.3
```

**EDA CODE BLOCK 2-Missing value analysis**

The second step focuses on characterizing missing data, including global missingness levels, missing patterns across groups and classes, and 10 variables that show the highest proportions of missing values.

```
introduce(df) %>%
  t()
##                          [,1]
## rows                     7403
## columns                   390
## discrete_columns            2
## continuous_columns        388
## all_missing_columns         0
## total_missing_values      497
## complete_rows            6924
## total_observations    2887170
## memory_usage         23835264
cat("\n")

row_missing <- rowSums(is.na(df))
col_missing <- colSums(is.na(df))

cat(" Rows with missing:", sum(row_missing > 0), "/", nrow(df),
    "(", round(mean(row_missing > 0)*100, 1), "%)\n")
##  Rows with missing: 479 / 7403 ( 6.5 %)
cat(" Cols with missing:", sum(col_missing > 0), "/", ncol(df),
    "(", round(mean(col_missing > 0)*100, 1), "%)\n\n")
##  Cols with missing: 271 / 390 ( 69.5 %)
cat("\n")

cat("Number of missing values in Class:", sum(is.na(df$Class)), "\n")
## Number of missing values in Class: 0
cat("Number of missing values in Info_PepID:", sum(is.na(df$Info_PepID)), "\n")
## Number of missing values in Info_PepID: 0
cat("Number of missing values in Info_pos:", sum(is.na(df$Info_pos)), "\n")
## Number of missing values in Info_pos: 1
```

```r
cat("Number of missing values in Info_group:", sum(is.na(df$Info_group)), "\n")
## Number of missing values in Info_group: 0
feat_cols <- names(df)[startsWith(names(df), "feat_")]
missing_feat <- sapply(df[feat_cols], function(x) sum(is.na(x)))
cat("Total missing values in feat_ columns:", sum(missing_feat), "\n")
## Total missing values in feat_ columns: 496
cat("\n")

if("Info_group" %in% names(df)) {
  group_pattern <- df %>%
    group_by(Info_group) %>%
    summarise(missing_rate = mean(rowSums(is.na(.)) > 0)) %>%
    filter(missing_rate > 0.1)

  if(nrow(group_pattern) > 0) {
    cat(" Groups with high missing:", nrow(group_pattern), "\n")
  }
}

class_pattern <- df %>%
  group_by(Class) %>%
  summarise(missing_rate = mean(rowSums(is.na(.)) > 0))
cat(" Class missing rates:", paste0(round(class_pattern$missing_rate*100, 1), "%"), "\n\n")
##  Class missing rates: 6.5% 6.5%

missing_top10 <- df %>%
  summarise(across(everything(), ~sum(is.na(.)))) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "missing_count") %>%
  filter(missing_count > 0) %>%
  mutate(
    missing_percent = round(missing_count/nrow(df)*100, 4),
    variable = fct_reorder(variable, missing_percent, .desc = TRUE)
  ) %>%
  arrange(desc(missing_percent)) %>%
  head(10)


ggplot(missing_top10, aes(x = variable, y = missing_percent)) +
  geom_col() +
  coord_flip() +
  labs(title = "Top 10 variables with highest missing percentages",
       x = "Variable", y = "Missing Percentage (%)")
```




**EDA CODE BLOCK 3-Missing Value Handling**

In this code block, I removed the the row with missing info_pos data, and impute all other missing values using median imputation for numerical features. This process aims to prepare the dataset for more effective outlier detection in subsequent analysis.

```
df_imputed <- df

df_imputed <- df_imputed[!is.na(df_imputed$Info_pos), ]
cat(" Removed 1 row with missing Info_pos\n")
##  Removed 1 row with missing Info_pos

numeric_features <- grep("^feat_", names(df_imputed), value = TRUE)
df_imputed[numeric_features] <- df_imputed[numeric_features] %>%
  mutate(across(everything(), ~ifelse(is.na(.), median(., na.rm = TRUE), .)))

cat(" Remaining NAs:", sum(is.na(df_imputed)), "\n\n")
##  Remaining NAs: 0
```

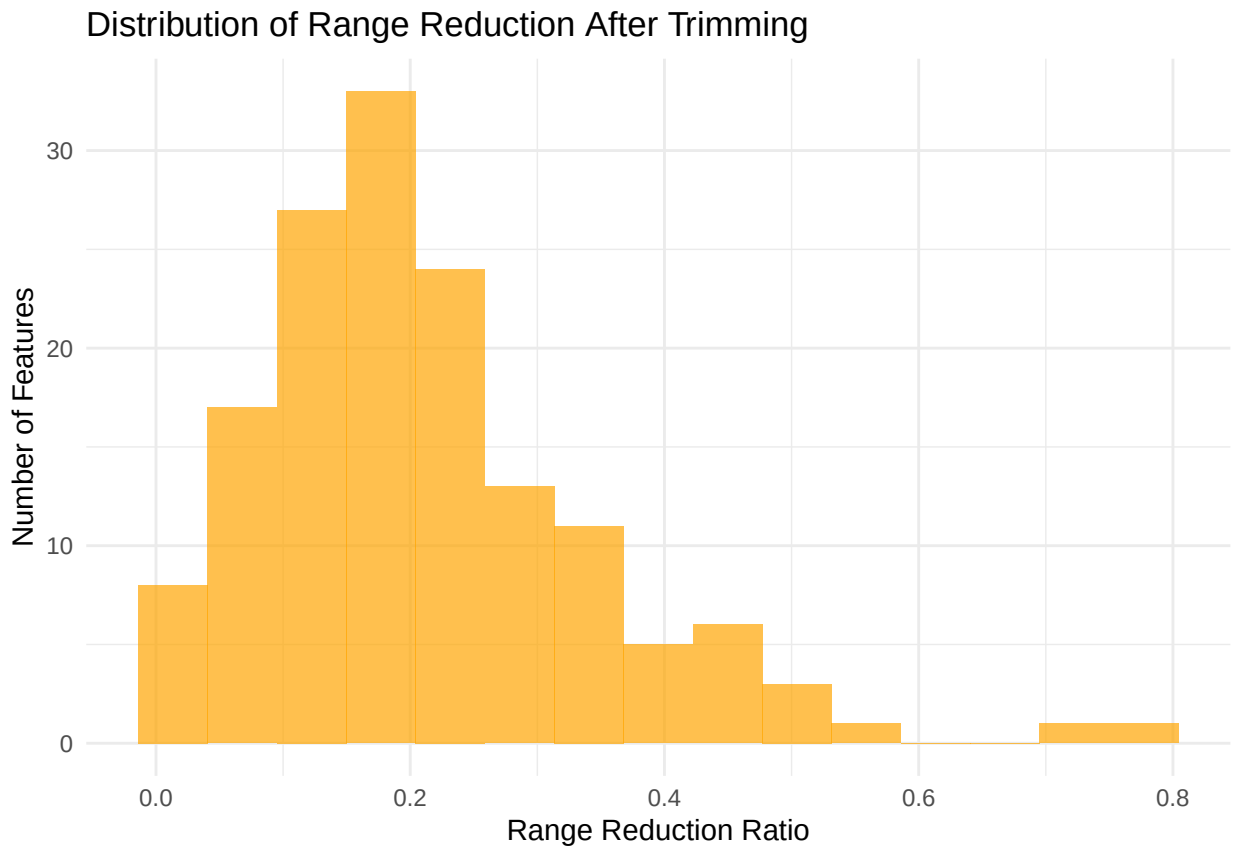
## EDA CODE BLOCK 4-Outlier Analysis

After that, I evaluate the impact of outliers on numerical features. I identified that some features require Winsorization in subsequent data processing. To visualize this, I plot the histogram of range reduction for

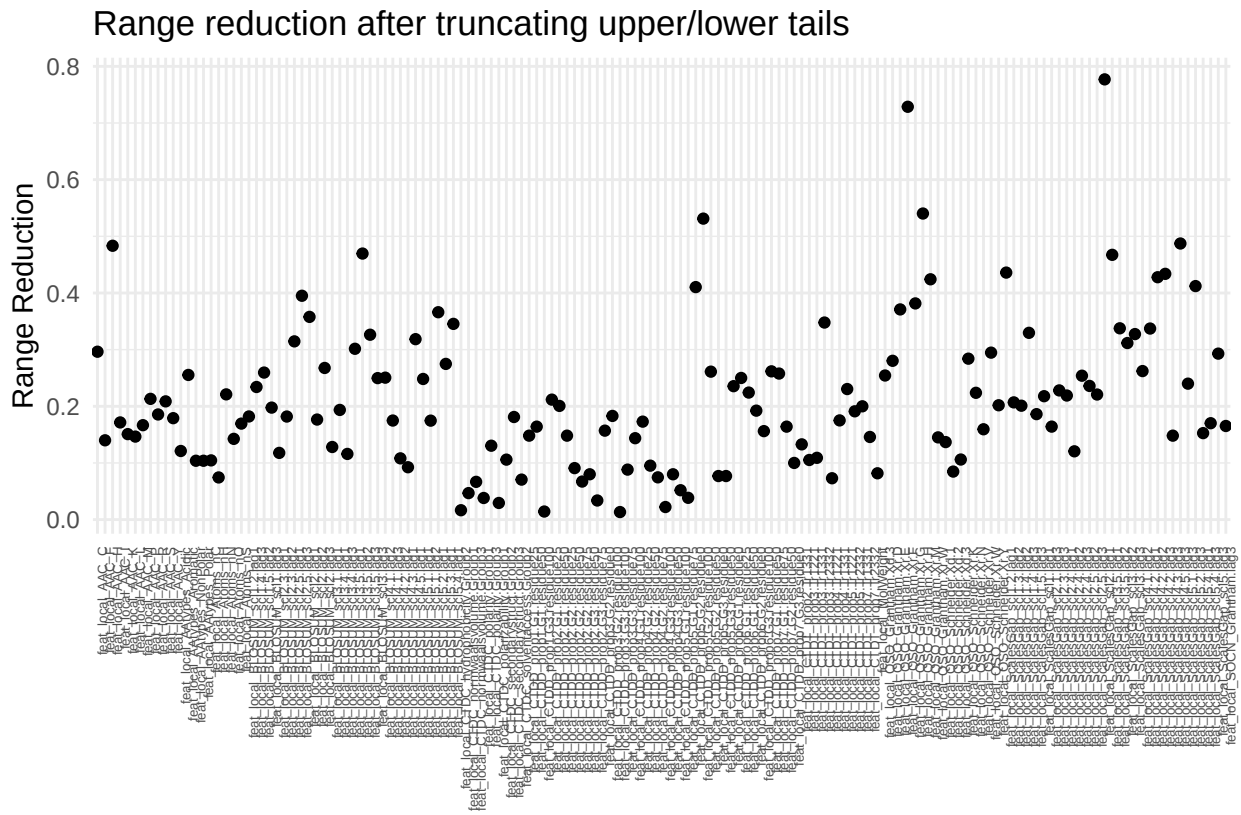
top 10 affected features and detailed dot plot of range reduction for sampled features. The analysis confirms significant outlier influence, justifying Winsorization treatment.

```
myrange <- function(x, trim = 0){
  n <- length(x)
  lo <- floor(n * trim) + 1
  hi <- n + 1 - lo
  return(diff(range(sort.int(x)[lo:hi])))
}

set.seed(MY_STUDENT_ID)
sampled_features <- sample(grep("^feat_", names(df_imputed)), 150)

df_num <- df_imputed[sampled_features]
range_comparison <- data.frame(
  Var = names(df_num),
  Range.00 = apply(df_num, 2, myrange),
  Range.01 = apply(df_num, 2, \(x) myrange(x, 0.001))
) %>%
  mutate(RangeRed = 1 - Range.01 / Range.00)


top_range_reduction <- range_comparison %>%
  arrange(desc(RangeRed)) %>%
  head(10)

print(top_range_reduction)
##                                                                    Var
## feat_local_ScalesGap_scl2.lag1           feat_local_ScalesGap_scl2.lag1
## feat_local_QSO_Grantham.Xr.F               feat_local_QSO_Grantham.Xr.F
## feat_local_QSO_Grantham.Xr.H               feat_local_QSO_Grantham.Xr.H
## feat_local_CTDD_prop5.G2.residue100 feat_local_CTDD_prop5.G2.residue100
## feat_local_ScalesGap_scl4.5.lag3       feat_local_ScalesGap_scl4.5.lag3
## feat_local_AAC_H                                     feat_local_AAC_H
## feat_local_BLOSUM_scl3.5.lag2             feat_local_BLOSUM_scl3.5.lag2
## feat_local_ScalesGap_scl3.5.lag1       feat_local_ScalesGap_scl3.5.lag1
## feat_local_ScalesGap_scl1.2.lag1       feat_local_ScalesGap_scl1.2.lag1
## feat_local_ScalesGap_scl4.2.lag3       feat_local_ScalesGap_scl4.2.lag3
##                                          Range.00     Range.01  RangeRed
## feat_local_ScalesGap_scl2.lag1      2.164694e+02 4.827270e+01 0.7770000
## feat_local_QSO_Grantham.Xr.F        4.743020e-04 1.287107e-04 0.7286314
## feat_local_QSO_Grantham.Xr.H        3.545421e-04 1.630042e-04 0.5402404
## feat_local_CTDD_prop5.G2.residue100 2.666667e+01 1.250000e+01 0.5312500
## feat_local_ScalesGap_scl4.5.lag3    1.431814e+02 7.340475e+01 0.4873304
## feat_local_AAC_H                    3.962963e-01 2.047619e-01 0.4833111
## feat_local_BLOSUM_scl3.5.lag2       1.469072e-01 7.792445e-02 0.4695670
## feat_local_ScalesGap_scl3.5.lag1    1.759682e+02 9.373792e+01 0.4673019
## feat_local_ScalesGap_scl1.2.lag1    1.582891e+02 8.928070e+01 0.4359643
## feat_local_ScalesGap_scl4.2.lag3    8.517722e+01 4.823055e+01 0.4337623
cat("\n")

ggplot(range_comparison, aes(x = RangeRed)) +
  geom_histogram(bins = 15, fill = "orange", alpha = 0.7) +
  labs(title = "Distribution of Range Reduction After Trimming",
```

```
       x = "Range Reduction Ratio", y = "Number of Features") +
  theme_minimal()
```




```
ggplot(range_comparison, aes(x = Var, y = RangeRed)) +
  geom_point() +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 90, hjust = 1, size = 5)) +
  labs(title = "Range reduction after truncating upper/lower tails ",
       x = "", y = "Range Reduction")
```



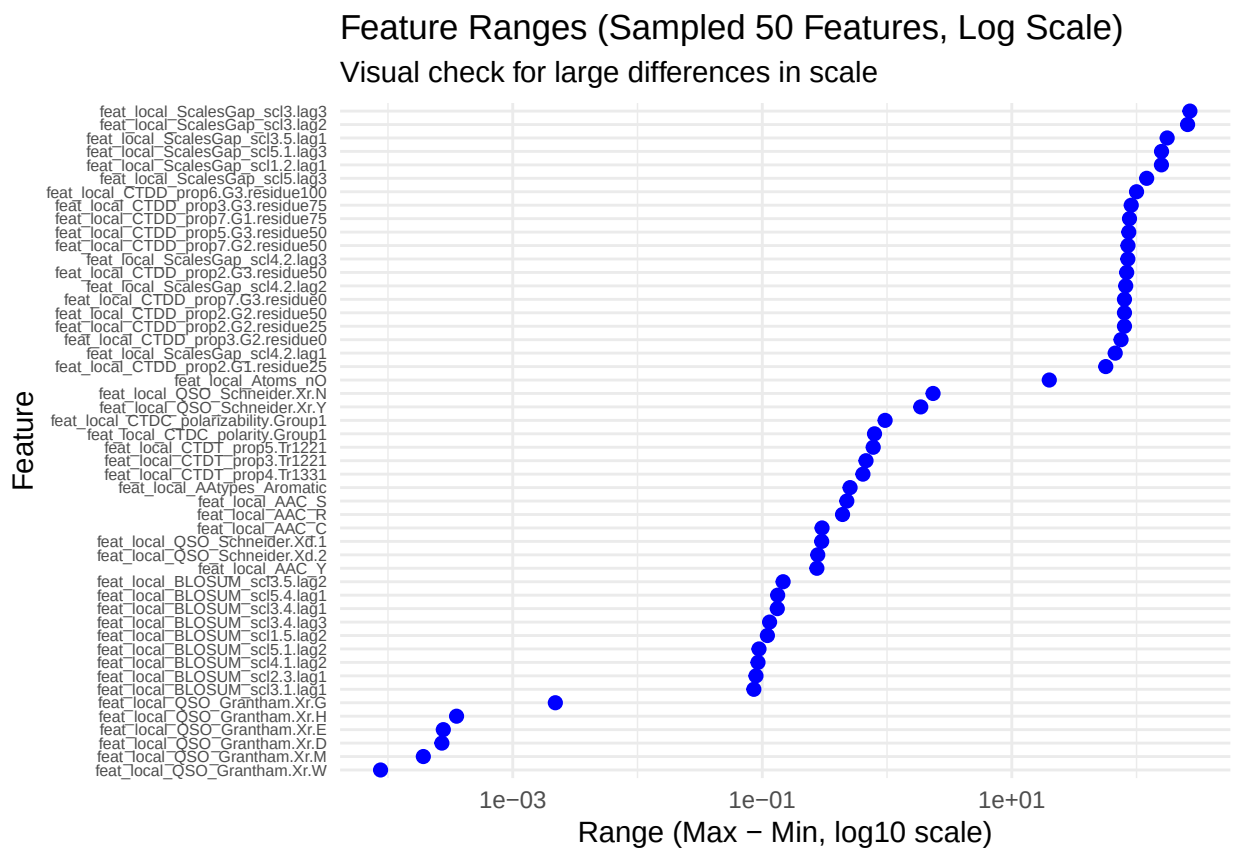

```
cat("Outlier Impact Summary:\n")
## Outlier Impact Summary:
avg_range_reduction <- mean(range_comparison$RangeRed)
max_range_reduction <- max(range_comparison$RangeRed)
features_high_reduction <- sum(range_comparison$RangeRed > 0.5)

cat(" Average range reduction:", round(avg_range_reduction, 3), "\n")
##  Average range reduction: 0.212
cat(" Maximum range reduction:", round(max_range_reduction, 3), "\n")
##  Maximum range reduction: 0.777
cat(" Features with >50% range reduction:", features_high_reduction, "\n\n")
##  Features with >50% range reduction: 4
```

**EDA CODE BLOCK 5-Feature Scale Analysis**

In this step, I randomly sample 50 feat features for scale analysis and visualize their value ranges on a logarithmic scale. Results show extremely large range ratios, indicating severe scale differences among features. This finding shows the necessity for feature standardization in subsequent processing steps.

```
set.seed(MY_STUDENT_ID)
sampled_features <- sample(grep("^feat_", names(df_imputed)), 50)

scale_data <- data.frame(
  feature = names(df_imputed)[sampled_features],
```

```r
  min_val = sapply(df_imputed[sampled_features], min, na.rm = TRUE),
  max_val = sapply(df_imputed[sampled_features], max, na.rm = TRUE),
  mean_val = sapply(df_imputed[sampled_features], mean, na.rm = TRUE)
) %>%
  mutate(range = max_val - min_val)

ggplot(scale_data, aes(x = reorder(feature, range), y = range)) +
  geom_point(size = 2, color = "blue") +
  scale_y_log10() +
  coord_flip() +
  labs(
    title = "Feature Ranges (Sampled 50 Features, Log Scale)",
    subtitle = "Visual check for large differences in scale",
    x = "Feature",
    y = "Range (Max - Min, log10 scale)"
  ) +
  theme_minimal() +
  theme(axis.text.y = element_text(size = 6))
```




```r
cat("Scale Difference Statistics:\n")
## Scale Difference Statistics:
scale_summary <- scale_data %>%
  summarise(
    min_range = min(range),
```

```r
    max_range = max(range),
    range_ratio = max_range / min_range
  )

cat(" Minimum range:", round(scale_summary$min_range, 4), "\n")
##  Minimum range: 1e-04
cat(" Maximum range:", round(scale_summary$max_range, 2), "\n")
##  Maximum range: 267.76
cat(" Range ratio:", round(scale_summary$range_ratio, 2), "\n\n")
##  Range ratio: 3069752
```

### EDA CODE BLOCK 6-Duplicate Check

At last, I check for completely identical rows and duplicate ID combinations in the dataset to prevent potential data collection errors that could interfere with model training and lead to overfitting. The check results confirm the uniqueness of the data.

```r
cat("1. Exact Duplicate Rows:\n")
## 1. Exact Duplicate Rows:
exact_duplicates <- df_imputed[duplicated(df_imputed), ]
total_exact_duplicates <- nrow(exact_duplicates)

if(total_exact_duplicates > 0) {
  cat("Found", total_exact_duplicates, "completely duplicate rows\n")
  cat("First 5 duplicate rows:\n")
  print(head(exact_duplicates, 5))
} else {
  cat("No completely duplicate rows found\n")
}
## No completely duplicate rows found
cat("\n")

cat("2. ID-based Duplicate Check:\n")
## 2. ID-based Duplicate Check:
id_duplicates <- df_imputed %>%
  count(Info_PepID, Info_pos) %>%
  filter(n > 1)

if(nrow(id_duplicates) > 0) {
  cat("Found", nrow(id_duplicates), "duplicate ID combinations\n")
  cat("Sample of duplicate ID combinations:\n")
  print(head(id_duplicates, 5))

} else {
  cat("No duplicate ID combinations found\n")
}
## No duplicate ID combinations found
```

---

## 2. Data preprocessing and feature engineering

**Note**: It is expected that this section should result in no more than 400-ish words (possibly less; not counting the code blocks).

Add here your data pre-processing and feature selection/engineering steps. Your solution should include both your code (which markers must be able to run independently) and your rationale. In general, your steps should include:

- A short justification of which insight from your EDA is motivating a given data transformation.
- A code block implementing your pre-processing step.

**In addition** to the regular data transformations emerging from the insights you gathered during EDA, this section must contain the splitting off of a test set that you'll use later for final performance assessment (check the CW specs for details).

At the end of this section you should also include a short commentary on what changes the pre-processing has induced in your data (i.e., what did the data "look like" before this step vs. what it "looks like" afterwards.

You can use as many code blocks as needed, but don't overdo it (remember - everything needs to have a proper rationale). If you compare different pre-processing options, make sure to make it clear, and to indicate at the end which option you selected, and why.

**Note**: In this section you'll be basically defining which data transformations you will later incorporate into your predictive pipeline. When you build the pipeline in the next section, you'll encapsulate all those steps (together with the model) into a single object that you can fit and deploy as a single unit.

### Data processing CODE BLOCK 1-Dataset Splitting

To prevent data leakage, I use the group_initial_split function to split the dataset into training and test sets(8:2). I verify group integrity by ensuring no overlapping groups between the splits, thus maintaining the validity of the separation and preventing data from the same group from being divided across both sets.

```r
set.seed(MY_STUDENT_ID)
split <- group_initial_split(df, group = Info_group, prop = 0.8)
df_train <- training(split)
df_test <- testing(split)

cat("Data splitting completed:\n")
## Data splitting completed:
cat(" Training set:", nrow(df_train), "observations\n")
##  Training set: 5857 observations
cat(" Test set:", nrow(df_test), "observations\n")
##  Test set: 1546 observations
cat(" Total groups in training:", n_distinct(df_train$Info_group), "\n")
##  Total groups in training: 397
cat(" Total groups in test:", n_distinct(df_test$Info_group), "\n\n")
##  Total groups in test: 153

cat("Group integrity check:\n")
## Group integrity check:
training_groups <- unique(df_train$Info_group)
test_groups <- unique(df_test$Info_group)
common_groups <- intersect(training_groups, test_groups)
```

```r
if(length(common_groups) == 0) {
  cat("SUCCESS: No groups overlap between training and test sets\n")
} else {
  cat("WARNING:", length(common_groups), "groups appear in both sets\n")
}
## SUCCESS: No groups overlap between training and test sets
```

### Data processing CODE BLOCK 2-Preprocessing

In this code block, I construct two preprocessing pipelines (data_recipe and rf_recipe) tailored to different algorithm characteristics. The rf_recipe pipeline is designed for Random Forest, excludes PCA steps. Both pipelines include data cleaning and missing value imputation using trimmed means. Besides,to address class imbalance in the training data, I calculate class weights: Class 1 weight: 1.384; Class -1 weight: 1.000. These weights will be passed to algorithms during model training to improve minority class recognition.

```r
df_train <- df_train %>%
  mutate(Class = as.factor(Class))
df_test <- df_test %>%
  mutate(Class = as.factor(Class))

data_recipe <- recipe(Class ~ ., data = df_train) %>%
  step_rm(matches("^Info_(?!group$)", perl = TRUE)) %>%
  update_role(Info_group, new_role = "group")%>%
  step_impute_mean(all_numeric_predictors(), trim = 0.1) %>%
  step_hai_winsorized_truncate(all_numeric_predictors(), fraction = 0.1) %>%
  step_normalize(all_numeric_predictors()) %>%
  step_pca(all_numeric_predictors(), threshold = 0.95)

rf_recipe <- recipe(Class ~ ., data = df_train) %>%
  step_rm(matches("^Info_(?!group$)", perl = TRUE)) %>%
  update_role(Info_group, new_role = "group")%>%
  step_impute_mean(all_numeric_predictors(), trim = 0.02) %>%
  step_hai_winsorized_truncate(all_numeric_predictors(), fraction = 0.05) %>%
  step_normalize(all_numeric_predictors())

class_counts <- table(df_train$Class)
class_weights <- c(
  "1" = as.numeric(max(class_counts) / class_counts["1"]),
  "-1" = as.numeric(max(class_counts) / class_counts["-1"])
)
```

### Data processing CODE BLOCK 3-Preprocessing Validation

At the end of data preprocessing, I apply both data_recipe and rf_recipe to verify their effectiveness. Results show that data_recipe successfully eliminates missing values, reduces features from 385 to 167 while achieving 95% variance explanation, and centers principal components. For rf_recipe, all missing values are handled and feature standardization achieves mean=0 and standard deviation=1. Both preprocessing pipelines are effective, confirming data quality satisfies modeling requirements for subsequent model selection and training.

```r
cat(" 1.data_recipe Validation \n\n")
##  1.data_recipe Validation

prep_data_recipe <- prep(data_recipe, training = df_train)
df_train_data <- bake(prep_data_recipe, new_data = df_train)
df_test_data <- bake(prep_data_recipe, new_data = df_test)

cat("data_recipe Basic Results:\n")
## data_recipe Basic Results:
cat(" Training set dimensions:", dim(df_train_data), "\n")
##  Training set dimensions: 5857 169
cat(" Test set dimensions:", dim(df_test_data), "\n")
##  Test set dimensions: 1546 169
cat(" Principal components:", sum(grepl("^PC", names(df_train_data))), "\n\n")
##  Principal components: 167

cat("Missing Value Treatment:\n")
## Missing Value Treatment:
missing_before_data <- sum(is.na(df_train[grep("^feat_", names(df_train))]))
missing_after_data <- sum(is.na(df_train_data))
cat(" Before preprocessing:", missing_before_data, "\n")
##  Before preprocessing: 409
cat(" After preprocessing:", missing_after_data, "\n")
##  After preprocessing: 0
cat(" Missing value resolution:", ifelse(missing_after_data == 0, "COMPLETE", "INCOMPLETE"), "\n\n")
##  Missing value resolution: COMPLETE

cat("Feature Scaling Validation:\n")
## Feature Scaling Validation:
pc_cols_data <- grep("^PC", names(df_train_data))
if(length(pc_cols_data) > 0) {
  pc_means_data <- sapply(df_train_data[pc_cols_data], mean, na.rm = TRUE)
  pc_sds_data <- sapply(df_train_data[pc_cols_data], sd, na.rm = TRUE)
  cat(" Mean of PC means:", round(mean(abs(pc_means_data)), 6), "(expected near 0)\n")
  cat(" Mean of PC standard deviations:", round(mean(pc_sds_data), 4), "(expected near 1)\n")
  cat(" Scaling status:", ifelse(mean(pc_sds_data) > 0.9 & mean(pc_sds_data) < 1.1, "SUCCESS", "CHECK N
}
##  Mean of PC means: 0 (expected near 0)
##  Mean of PC standard deviations: 1.7272 (expected near 1)
##  Scaling status: CHECK NEEDED
cat("\n")

cat("Dimensionality Reduction Analysis:\n")
## Dimensionality Reduction Analysis:
cat(" Original numeric features: 385\n")
##  Original numeric features: 385
cat(" Principal components retained:", sum(grepl("^PC", names(df_train_data))), "\n")
##  Principal components retained: 167
reduction_ratio_data <- (385 - sum(grepl("^PC", names(df_train_data)))) / 385 * 100
cat(" Feature reduction:", round(reduction_ratio_data, 1), "%\n")
##  Feature reduction: 56.6 %

pca_step_data <- prep_data_recipe$steps[[5]]
```

```r
if(!is.null(pca_step_data$res)) {
  variance_explained_data <- pca_step_data$res$sdev^2 / sum(pca_step_data$res$sdev^2)
  cumulative_variance_data <- cumsum(variance_explained_data)
  actual_variance_data <- cumulative_variance_data[sum(grepl("^PC", names(df_train_data)))]
  cat(" Cumulative variance explained:", round(actual_variance_data * 100, 1), "%\n")
  cat(" Target variance: 95%\n")
  cat(" PCA status:", ifelse(actual_variance_data >= 0.95, "TARGET ACHIEVED", "BELOW TARGET"), "\n")
}
##  Cumulative variance explained: 95 %
##  Target variance: 95%
##  PCA status: TARGET ACHIEVED
cat("\n")

cat(" 2.rf_recipe Validation \n\n")
##  2.rf_recipe Validation

prep_rf_recipe <- prep(rf_recipe, training = df_train, retain = TRUE)
df_train_rf <- bake(prep_rf_recipe, new_data = NULL)
df_test_rf <- bake(prep_rf_recipe, new_data = df_test)

cat("rf_recipe Basic Results:\n")
## rf_recipe Basic Results:
cat(" Training set dimensions:", dim(df_train_rf), "\n")
##  Training set dimensions: 5857 772
cat(" Test set dimensions:", dim(df_test_rf), "\n")
##  Test set dimensions: 1546 772
cat(" Numeric features retained:", sum(grepl("^feat_", names(df_train_rf))), "\n\n")
##  Numeric features retained: 385

cat("Missing Value Treatment:\n")
## Missing Value Treatment:
missing_before_rf <- sum(is.na(df_train[grep("^feat_", names(df_train))]))
missing_after_rf <- sum(is.na(df_train_rf))
cat(" Before preprocessing:", missing_before_rf, "\n")
##  Before preprocessing: 409
cat(" After preprocessing:", missing_after_rf, "\n")
##  After preprocessing: 0
cat(" Missing value resolution:", ifelse(missing_after_rf == 0, "COMPLETE", "INCOMPLETE"), "\n\n")
##  Missing value resolution: COMPLETE

cat("Feature Standardization Validation:\n")
## Feature Standardization Validation:
feat_cols_rf <- grep("^feat_", names(df_train_rf))
if(length(feat_cols_rf) > 0) {
  feat_means_rf <- sapply(df_train_rf[feat_cols_rf], mean, na.rm = TRUE)
  feat_sds_rf <- sapply(df_train_rf[feat_cols_rf], sd, na.rm = TRUE)
  cat(" Mean of feature means:", round(mean(abs(feat_means_rf)), 6), "(expected near 0)\n")
  cat(" Mean of feature standard deviations:", round(mean(feat_sds_rf), 4), "(expected near 1)\n")
  cat(" Standardization status:", ifelse(mean(feat_sds_rf) > 0.9 & mean(feat_sds_rf) < 1.1, "SUCCESS",
}
##  Mean of feature means: 0 (expected near 0)
##  Mean of feature standard deviations: 1 (expected near 1)
##  Standardization status: SUCCESS
```

```r
cat("\n")

cat("Data Integrity Check:\n")
## Data Integrity Check:
cat("- Remaining NA in training:", sum(is.na(df_train_rf)), "\n")
## - Remaining NA in training: 0
cat("- Remaining NA in test:", sum(is.na(df_test_rf)), "\n")
## - Remaining NA in test: 0
cat("- Class balance in training:", table(df_train_rf$Class), "\n")
## - Class balance in training: 3400 2457
cat("- Class balance in test:", table(df_test_rf$Class), "\n")
## - Class balance in test: 942 604
cat("- Data integrity:", ifelse(sum(is.na(df_train_rf)) == 0 & sum(is.na(df_test_rf)) == 0, "MAINTAINED
## - Data integrity: MAINTAINED
```

---

## 3. Modelling

**Note**: It is expected that this section should result in no more than 400-ish words (possibly less; not counting the code blocks).

Add here your modelling steps. Your solution should include both your code (which markers must be able to run independently) and your rationale. This section must include:

- The fitting of a preliminary model on your preprocessed training data (to estimate some baseline performance level and to make sure your data is ready for modelling). Make this a simple model - logistic regression, decision trees or kNN are good options (but other may be used instead, if you prefer).
- The building of a predictive pipeline encapsulating all the learned/learnable aspects of your solution. This should result in a pipeline-type object that uses your raw training data (post EDA and test set splitting, but prior to any preprocessing), fits all your preprocessing transformations (incl. dimensionality reduction) and the predictive model using that data, and can then be used to ingest your test data (or any new data in the same format) and generate predictions for each entry.

Please refer to the CW specs for other requirements of this step (including, e.g., the addition of a dimensionality reduction step into your pipeline).

You can use as many code blocks as needed, but don't overdo it (remember - everything needs to have a proper rationale). If you compare different models, make sure to make it clear, and to indicate at the end which pipeline option is your final selected one.

### Modelling CODE BLOCK 1-Cross-Validation Setup

At fist, I establish a grouped cross-validation scheme for subsequent model evaluation and hyperparameter tuning.

```r
set.seed(MY_STUDENT_ID)
cv_folds <- group_vfold_cv(df_train, v = 3, group = Info_group)
```

### Modelling CODE BLOCK 2-Baseline Model Establishment

Then, I establish a logistic regression model as a performance benchmark. The model utilizes the data_recipe preprocessing pipeline and is evaluated using 3-fold grouped cross-validation, with results reported for F1 score, accuracy, and ROC AUC. The purposes of this baseline model are: (1)To provide a performance reference for subsequent complex models (2)To verify the proper functioning of the preprocessing pipeline (3)To establish a quantitative performance starting point for future improvements Results show a current F1 Score: 0.7281

```
logreg_model <- logistic_reg() %>%
  set_engine("glm") %>%
  set_mode("classification")

baseline_wf <- workflow() %>%
  add_recipe(data_recipe) %>%
  add_model(logreg_model)

baseline_cv <- fit_resamples(
  baseline_wf,
  resamples = cv_folds,
  metrics = metric_set(f_meas, accuracy, roc_auc)
)

baseline_metrics <- collect_metrics(baseline_cv)

cat("Baseline Model Cross-Validation Results:\n")
## Baseline Model Cross-Validation Results:
cat(" F1 Score:", round(baseline_metrics$mean[baseline_metrics$.metric == "f_meas"], 4), "\n")
##  F1 Score: 0.7281
cat(" Accuracy:", round(baseline_metrics$mean[baseline_metrics$.metric == "accuracy"], 4), "\n")
##  Accuracy: 0.66
cat(" ROC AUC:", round(baseline_metrics$mean[baseline_metrics$.metric == "roc_auc"], 4), "\n\n")
##  ROC AUC: 0.6946
```

### Modelling CODE BLOCK 3-Model Comparison

In this code block, I compare the performance of complex models Random Forest and XGBoost, using fixed parameters. Both models are evaluated through 3-fold cross-validation to assess their preliminary performance. The objective is to select the best-performing model for subsequent in-depth hyperparameter tuning.

```
set.seed(MY_STUDENT_ID)

models_list <- list(
  "random_forest" = rand_forest(
    mtry = 20,
    trees = 100,
    min_n = 10
  ) %>%
    set_engine("ranger") %>%
    set_mode("classification"),

  "xgboost" = boost_tree(
```

```r
    trees = 100,
    learn_rate = 0.1,
    tree_depth = 6,
    min_n = 10
  ) %>%
    set_engine("xgboost") %>%
    set_mode("classification")
)

model_results <- list()

for (model_name in names(models_list)) {
  cat("Evaluation of:", model_name, "\n")

  tryCatch({
    if (model_name == "random_forest") {
      model_recipe <- rf_recipe
    } else {
      model_recipe <- data_recipe
    }

    model_wf <- workflow() %>%
      add_recipe(model_recipe) %>%
      add_model(models_list[[model_name]])

    cv_results <- fit_resamples(
      model_wf,
      resamples = cv_folds,
      metrics = metric_set(f_meas, roc_auc),
      control = control_resamples(save_pred = FALSE, verbose = FALSE)
    )

    metrics_df <- collect_metrics(cv_results)
    f1_val <- metrics_df$mean[metrics_df$.metric == "f_meas"]
    auc_val <- metrics_df$mean[metrics_df$.metric == "roc_auc"]

    model_results[[model_name]] <- data.frame(
      model = model_name,
      f1_score = f1_val,
      auc = auc_val
    )

    cat("Model", model_name, "completed - F1:", round(f1_val, 4), "\n")

  }, error = function(e) {
    cat("Model", model_name, "evaluation failed:", e$message, "\n")
    model_results[[model_name]] <- data.frame(
      model = model_name,
      f1_score = NA,
      auc = NA
    )
  })
}
```

```
## Evaluation of: random_forest
## Model random_forest completed - F1: 0.7649
## Evaluation of: xgboost
## Model xgboost completed - F1: 0.7507

if(any(!is.na(model_results$random_forest$f1_score) | !is.na(model_results$xgboost$f1_score))) {

  rf_f1 <- ifelse(!is.na(model_results$random_forest$f1_score), model_results$random_forest$f1_score, 0
  xgb_f1 <- ifelse(!is.na(model_results$xgboost$f1_score), model_results$xgboost$f1_score, 0)

  if(rf_f1 >= xgb_f1) {
    best_model_name <- "random_forest"
    best_f1 <- rf_f1
  } else {
    best_model_name <- "xgboost"
    best_f1 <- xgb_f1
  }

  cat("\nSELECTED FOR DETAILED TUNING:", best_model_name, "(F1:", round(best_f1, 4), ")\n")

} else {
  cat("\nERROR\n")
  best_model_name <- NULL
}
##
## SELECTED FOR DETAILED TUNING: random_forest (F1: 0.7649 )
```

### Modelling CODE BLOCK 4- Random Forest Model

Based on the previous comparison, I found out that the Random Forest model demonstrated the best performance, so I select it as my final model. In this section, I define the tunable Random Forest model to prepare for subsequent hyperparameter optimization steps.

```
rf_model <- rand_forest(
  mtry = tune(),
  trees = tune(),
  min_n = tune()
) %>%
  set_engine("ranger",class.weights = class_weights) %>%
  set_mode("classification")

rf_wf <- workflow() %>%
  add_recipe(rf_recipe) %>%
  add_model(rf_model)
```

### Modelling CODE BLOCK 5-Hyperparameter Tuning

Before the final evaluation, I perform systematic hyperparameter tuning for the Random Forest model using grid search. I explore optimal combinations of three parameters: mtry, trees, and min_n. A regular grid of 27 parameter combinations is created and evaluated using multiple metrics. The tuning results display the top 5 performing parameter configurations, providing the basis for final model selection.

```r
rf_grid <- grid_regular(
  mtry(range = c(8, 30)),
  trees(range = c(300, 1000)),
  min_n(range = c(5, 20)),
  levels = 3
)

cat("Using optimized grid with only", nrow(rf_grid), "combinations\n")
## Using optimized grid with only 27 combinations
rf_tune <- tune_grid(
  rf_wf,
  resamples = cv_folds,
  grid = rf_grid,
  metrics = metric_set(f_meas, roc_auc, accuracy, precision, recall),
  control = control_grid(verbose = TRUE, save_pred = TRUE)
)
## i Resample1: preprocessor 1/1
## i Resample1: preprocessor 1/1, model 1/27
## i Resample1: preprocessor 1/1, model 1/27 (predictions)
## i Resample1: preprocessor 1/1, model 2/27
## i Resample1: preprocessor 1/1, model 2/27 (predictions)
## i Resample1: preprocessor 1/1, model 3/27
## i Resample1: preprocessor 1/1, model 3/27 (predictions)
## i Resample1: preprocessor 1/1, model 4/27
## i Resample1: preprocessor 1/1, model 4/27 (predictions)
## i Resample1: preprocessor 1/1, model 5/27
## i Resample1: preprocessor 1/1, model 5/27 (predictions)
## i Resample1: preprocessor 1/1, model 6/27
## i Resample1: preprocessor 1/1, model 6/27 (predictions)
## i Resample1: preprocessor 1/1, model 7/27
## i Resample1: preprocessor 1/1, model 7/27 (predictions)
## i Resample1: preprocessor 1/1, model 8/27
## i Resample1: preprocessor 1/1, model 8/27 (predictions)
## i Resample1: preprocessor 1/1, model 9/27
## i Resample1: preprocessor 1/1, model 9/27 (predictions)
## i Resample1: preprocessor 1/1, model 10/27
## i Resample1: preprocessor 1/1, model 10/27 (predictions)
## i Resample1: preprocessor 1/1, model 11/27
## i Resample1: preprocessor 1/1, model 11/27 (predictions)
## i Resample1: preprocessor 1/1, model 12/27
## i Resample1: preprocessor 1/1, model 12/27 (predictions)
## i Resample1: preprocessor 1/1, model 13/27
## i Resample1: preprocessor 1/1, model 13/27 (predictions)
## i Resample1: preprocessor 1/1, model 14/27
## i Resample1: preprocessor 1/1, model 14/27 (predictions)
## i Resample1: preprocessor 1/1, model 15/27
## i Resample1: preprocessor 1/1, model 15/27 (predictions)
## i Resample1: preprocessor 1/1, model 16/27
## i Resample1: preprocessor 1/1, model 16/27 (predictions)
## i Resample1: preprocessor 1/1, model 17/27
## i Resample1: preprocessor 1/1, model 17/27 (predictions)
## i Resample1: preprocessor 1/1, model 18/27
## i Resample1: preprocessor 1/1, model 18/27 (predictions)
```

```
## i Resample1: preprocessor 1/1, model 19/27
## i Resample1: preprocessor 1/1, model 19/27 (predictions)
## i Resample1: preprocessor 1/1, model 20/27
## i Resample1: preprocessor 1/1, model 20/27 (predictions)
## i Resample1: preprocessor 1/1, model 21/27
## i Resample1: preprocessor 1/1, model 21/27 (predictions)
## i Resample1: preprocessor 1/1, model 22/27
## i Resample1: preprocessor 1/1, model 22/27 (predictions)
## i Resample1: preprocessor 1/1, model 23/27
## i Resample1: preprocessor 1/1, model 23/27 (predictions)
## i Resample1: preprocessor 1/1, model 24/27
## i Resample1: preprocessor 1/1, model 24/27 (predictions)
## i Resample1: preprocessor 1/1, model 25/27
## i Resample1: preprocessor 1/1, model 25/27 (predictions)
## i Resample1: preprocessor 1/1, model 26/27
## i Resample1: preprocessor 1/1, model 26/27 (predictions)
## i Resample1: preprocessor 1/1, model 27/27
## i Resample1: preprocessor 1/1, model 27/27 (predictions)
## i Resample2: preprocessor 1/1
## i Resample2: preprocessor 1/1, model 1/27
## i Resample2: preprocessor 1/1, model 1/27 (predictions)
## i Resample2: preprocessor 1/1, model 2/27
## i Resample2: preprocessor 1/1, model 2/27 (predictions)
## i Resample2: preprocessor 1/1, model 3/27
## i Resample2: preprocessor 1/1, model 3/27 (predictions)
## i Resample2: preprocessor 1/1, model 4/27
## i Resample2: preprocessor 1/1, model 4/27 (predictions)
## i Resample2: preprocessor 1/1, model 5/27
## i Resample2: preprocessor 1/1, model 5/27 (predictions)
## i Resample2: preprocessor 1/1, model 6/27
## i Resample2: preprocessor 1/1, model 6/27 (predictions)
## i Resample2: preprocessor 1/1, model 7/27
## i Resample2: preprocessor 1/1, model 7/27 (predictions)
## i Resample2: preprocessor 1/1, model 8/27
## i Resample2: preprocessor 1/1, model 8/27 (predictions)
## i Resample2: preprocessor 1/1, model 9/27
## i Resample2: preprocessor 1/1, model 9/27 (predictions)
## i Resample2: preprocessor 1/1, model 10/27
## i Resample2: preprocessor 1/1, model 10/27 (predictions)
## i Resample2: preprocessor 1/1, model 11/27
## i Resample2: preprocessor 1/1, model 11/27 (predictions)
## i Resample2: preprocessor 1/1, model 12/27
## i Resample2: preprocessor 1/1, model 12/27 (predictions)
## i Resample2: preprocessor 1/1, model 13/27
## i Resample2: preprocessor 1/1, model 13/27 (predictions)
## i Resample2: preprocessor 1/1, model 14/27
## i Resample2: preprocessor 1/1, model 14/27 (predictions)
## i Resample2: preprocessor 1/1, model 15/27
## i Resample2: preprocessor 1/1, model 15/27 (predictions)
## i Resample2: preprocessor 1/1, model 16/27
## i Resample2: preprocessor 1/1, model 16/27 (predictions)
## i Resample2: preprocessor 1/1, model 17/27
## i Resample2: preprocessor 1/1, model 17/27 (predictions)
```

```
## i Resample2: preprocessor 1/1, model 18/27
## i Resample2: preprocessor 1/1, model 18/27 (predictions)
## i Resample2: preprocessor 1/1, model 19/27
## i Resample2: preprocessor 1/1, model 19/27 (predictions)
## i Resample2: preprocessor 1/1, model 20/27
## i Resample2: preprocessor 1/1, model 20/27 (predictions)
## i Resample2: preprocessor 1/1, model 21/27
## i Resample2: preprocessor 1/1, model 21/27 (predictions)
## i Resample2: preprocessor 1/1, model 22/27
## i Resample2: preprocessor 1/1, model 22/27 (predictions)
## i Resample2: preprocessor 1/1, model 23/27
## i Resample2: preprocessor 1/1, model 23/27 (predictions)
## i Resample2: preprocessor 1/1, model 24/27
## i Resample2: preprocessor 1/1, model 24/27 (predictions)
## i Resample2: preprocessor 1/1, model 25/27
## i Resample2: preprocessor 1/1, model 25/27 (predictions)
## i Resample2: preprocessor 1/1, model 26/27
## i Resample2: preprocessor 1/1, model 26/27 (predictions)
## i Resample2: preprocessor 1/1, model 27/27
## i Resample2: preprocessor 1/1, model 27/27 (predictions)
## i Resample3: preprocessor 1/1
## i Resample3: preprocessor 1/1, model 1/27
## i Resample3: preprocessor 1/1, model 1/27 (predictions)
## i Resample3: preprocessor 1/1, model 2/27
## i Resample3: preprocessor 1/1, model 2/27 (predictions)
## i Resample3: preprocessor 1/1, model 3/27
## i Resample3: preprocessor 1/1, model 3/27 (predictions)
## i Resample3: preprocessor 1/1, model 4/27
## i Resample3: preprocessor 1/1, model 4/27 (predictions)
## i Resample3: preprocessor 1/1, model 5/27
## i Resample3: preprocessor 1/1, model 5/27 (predictions)
## i Resample3: preprocessor 1/1, model 6/27
## i Resample3: preprocessor 1/1, model 6/27 (predictions)
## i Resample3: preprocessor 1/1, model 7/27
## i Resample3: preprocessor 1/1, model 7/27 (predictions)
## i Resample3: preprocessor 1/1, model 8/27
## i Resample3: preprocessor 1/1, model 8/27 (predictions)
## i Resample3: preprocessor 1/1, model 9/27
## i Resample3: preprocessor 1/1, model 9/27 (predictions)
## i Resample3: preprocessor 1/1, model 10/27
## i Resample3: preprocessor 1/1, model 10/27 (predictions)
## i Resample3: preprocessor 1/1, model 11/27
## i Resample3: preprocessor 1/1, model 11/27 (predictions)
## i Resample3: preprocessor 1/1, model 12/27
## i Resample3: preprocessor 1/1, model 12/27 (predictions)
## i Resample3: preprocessor 1/1, model 13/27
## i Resample3: preprocessor 1/1, model 13/27 (predictions)
## i Resample3: preprocessor 1/1, model 14/27
## i Resample3: preprocessor 1/1, model 14/27 (predictions)
## i Resample3: preprocessor 1/1, model 15/27
## i Resample3: preprocessor 1/1, model 15/27 (predictions)
## i Resample3: preprocessor 1/1, model 16/27
## i Resample3: preprocessor 1/1, model 16/27 (predictions)
```

```
## i Resample3: preprocessor 1/1, model 17/27
## i Resample3: preprocessor 1/1, model 17/27 (predictions)
## i Resample3: preprocessor 1/1, model 18/27
## i Resample3: preprocessor 1/1, model 18/27 (predictions)
## i Resample3: preprocessor 1/1, model 19/27
## i Resample3: preprocessor 1/1, model 19/27 (predictions)
## i Resample3: preprocessor 1/1, model 20/27
## i Resample3: preprocessor 1/1, model 20/27 (predictions)
## i Resample3: preprocessor 1/1, model 21/27
## i Resample3: preprocessor 1/1, model 21/27 (predictions)
## i Resample3: preprocessor 1/1, model 22/27
## i Resample3: preprocessor 1/1, model 22/27 (predictions)
## i Resample3: preprocessor 1/1, model 23/27
## i Resample3: preprocessor 1/1, model 23/27 (predictions)
## i Resample3: preprocessor 1/1, model 24/27
## i Resample3: preprocessor 1/1, model 24/27 (predictions)
## i Resample3: preprocessor 1/1, model 25/27
## i Resample3: preprocessor 1/1, model 25/27 (predictions)
## i Resample3: preprocessor 1/1, model 26/27
## i Resample3: preprocessor 1/1, model 26/27 (predictions)
## i Resample3: preprocessor 1/1, model 27/27
## i Resample3: preprocessor 1/1, model 27/27 (predictions)
cat("\n Tuning Results \n")
##
##  Tuning Results
cat("Top 5 parameter combinations by F1 score:\n")
## Top 5 parameter combinations by F1 score:
print(show_best(rf_tune, metric = "f_meas", n = 5))
## # A tibble: 5 x 9
##    mtry trees min_n .metric .estimator  mean     n std_err .config
##   <int> <int> <int> <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1    30  1000     5 f_meas  binary     0.771     3  0.0334 pre0_mod25_post0
## 2    30   650     5 f_meas  binary     0.770     3  0.0327 pre0_mod22_post0
## 3    19  1000     5 f_meas  binary     0.770     3  0.0350 pre0_mod16_post0
## 4    30   300    12 f_meas  binary     0.770     3  0.0331 pre0_mod20_post0
## 5    19   650     5 f_meas  binary     0.769     3  0.0326 pre0_mod13_post0
```

### Modelling CODE BLOCK 6-Model Selection and Final Evaluation

This code block completes the final stage of model selection, including selecting the best parameters, training the final model, performing threshold optimization, and evaluating performance on the test set. It selects the parameter combination with the highest F1 score from the tuning results and uses these optimal parameters to train the final random forest model on the entire training set. Additionally, threshold optimization is performed to search for the optimal classification threshold.

```
best_params <- show_best(rf_tune, metric = "f_meas", n = 1)
print(best_params)
## # A tibble: 1 x 9
##    mtry trees min_n .metric .estimator  mean     n std_err .config
##   <int> <int> <int> <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1    30  1000     5 f_meas  binary     0.771     3  0.0334 pre0_mod25_post0
```

```r
best_rf <- select_best(rf_tune, metric = "f_meas")
final_rf_wf <- finalize_workflow(rf_wf, best_rf)

cv_predictions <- collect_predictions(rf_tune) %>%
  filter(.config == best_rf$.config)

balanced_optimization <- function(probs, true_labels) {
  thresholds <- seq(0.55, 0.75, 0.01)
  results <- map_df(thresholds, function(thresh) {
    pred_class <- ifelse(probs$.pred_1 > thresh, "1", "-1")
    f1 <- f_meas_vec(truth = true_labels, estimate = factor(pred_class, levels = c("-1", "1")))
    accuracy <- accuracy_vec(truth = true_labels, estimate = factor(pred_class, levels = c("-1", "1")))

    academic_score <- 0.7 * f1 + 0.3 * accuracy
    data.frame(threshold = thresh, f1 = f1, accuracy = accuracy, score = academic_score)
  })

  return(results[which.max(results$score), ])
}

academic_opt <- balanced_optimization(cv_predictions, cv_predictions$Class)
cat("BEST Threshold (from CV):", academic_opt$threshold, "\n")
## BEST Threshold (from CV): 0.55
cat("CV F1:", round(academic_opt$f1, 4), "\n")
## CV F1: 0.7703
cat("CV Accuracy:", round(academic_opt$accuracy, 4), "\n")
## CV Accuracy: 0.698

final_rf_fit <- fit(final_rf_wf, data = df_train)
```

### Modelling CODE BLOCK 7-Conclude

Finally, this code block shows the cross-validation performance comparison between the final model and the baseline model, quantifying the improvement achieved through the modeling pipeline. The results demonstrate that after systematic feature engineering, preprocessing, and hyperparameter tuning, the Random Forest model achieves a 6.7% improvement in F1-score compared to the simple Logistic Regression baseline.

```r
cat("\nModel Selection Summary:\n")
##
## Model Selection Summary:
cat(" Baseline Logistic Regression F1 (from CV): 0.7281\n")
##  Baseline Logistic Regression F1 (from CV): 0.7281
cat(" Tuned Random Forest F1 (from CV):", round(academic_opt$f1, 4), "\n")
##  Tuned Random Forest F1 (from CV): 0.7703
cat(" Accuracy:", round(academic_opt$accuracy, 4), "\n")
##  Accuracy: 0.698
cat(" Performance Improvement:", round(academic_opt$f1 - 0.7281, 4), "\n")
##  Performance Improvement: 0.0422
```

---

## 4. Model assessment

**Note**: It is expected that this section should result in no more than 100-ish words (possibly less; not counting the code blocks).

This is a short section in which you must estimate the generalisation performance of your model on the test set that you isolated at the start of your Data preprocessing step. It is also a chance for you to ensure that your pipeline object can take in new data and return the required predictions.

Use your pipeline to generate predictions for your test set, and calculate the observed performance using the $F_1$-score as your main performance indicator. You can also additionally report other performance metrics if you think it's relevant.

You can use as many code blocks as needed, but it's likely that this section can be completed with a single one. Make sure that your performance indicators are clearly echoed, so that a reader can easily check if after re-running your script. Add a short comment on the observed performance (is it good? Is it poor? Remember that you're not being assessed on the performance of your models, but on the soundness of your approach).

### Model Assessment CODE BLOCK

This code evaluates the final model's generalization performance on an independent test set. Results show a test set F1-score of 0.7737. This demonstrates the validates the effectiveness and rationality of the entire workflow from data preprocessing to model tuning.

```
test_probs <- predict(final_rf_fit, new_data = df_test, type = "prob")

test_predictions_optimized <- test_probs %>%
  mutate(.pred_class = ifelse(.pred_1 > academic_opt$threshold, "1", "-1")) %>%
  mutate(.pred_class = factor(.pred_class, levels = c("-1", "1")))

test_results <- df_test %>%
  select(Class) %>%
  bind_cols(test_predictions_optimized)

f1_test <- f_meas(test_results, truth = Class, estimate = .pred_class)
accuracy_test <- accuracy(test_results, truth = Class, estimate = .pred_class)
auc_test <- roc_auc(test_results, truth = Class, .pred_1)
precision_test <- precision(test_results, truth = Class, estimate = .pred_class)
recall_test <- recall(test_results, truth = Class, estimate = .pred_class)

cat("Generalization Performance on Test Set :\n")
## Generalization Performance on Test Set :
cat(" Optimal Threshold:", academic_opt$threshold, "\n")
##  Optimal Threshold: 0.55
cat(" F1 Score (main metric):", round(f1_test$.estimate, 4), "\n")
##  F1 Score (main metric): 0.7721
cat(" Accuracy:", round(accuracy_test$.estimate, 4), "\n")
##  Accuracy: 0.6915
cat(" Precision:", round(precision_test$.estimate, 4), "\n")
##  Precision: 0.702
cat(" Recall:", round(recall_test$.estimate, 4), "\n\n")
##  Recall: 0.8577
```

## 5. Model deployment

**Note**: This section is not expected to have much text in it, only the code and the results.

Finally, in this section you'll generate predictions for some data for which you do not know the class labels. This simulates the real-life scenario in which your model needs to be deployed to generate new predictions for new, unknown data (after all, if the labels were known, we wouldn't need a model).

Make sure that the data file **df_holdout.rds** is placed in the same folder as this Task02.Rmd file. The new data will then be automatically loaded and set up for you, and stored into a dataframe variable called `df_holdout`, with the same columns as the original `df` that was loaded at the start of this task.

- Use your fitted pipeline to generate predictions for this new data.
- Build a new data frame called containing the following columns:
  - `Info_PepID` (taken from df_holdout)
  - `Info_pos` (taken from df_holdout)
  - `Predicted_class` (with the predictions produced by your pipeline)
- Save your predictions dataframe to a file called **mypreds.rds**. You will be able to submit this file, and it will be used to give you some feedback on the actual performance of your pipeline on new data.

### Model deployment CODE BLOCK

In this final step, I deploy the fitted pipeline to generate predictions for the unseen holdout dataset and save the results for evaluation.

```
holdout_probs <- predict(final_rf_fit, new_data = df_holdout, type = "prob")

holdout_predictions_optimized <- holdout_probs %>%
  mutate(.pred_class = ifelse(.pred_1 > academic_opt$threshold, "1", "-1")) %>%
  mutate(.pred_class = factor(.pred_class, levels = c("-1", "1")))

predictions_df <- data.frame(
  Info_PepID = df_holdout$Info_PepID,
  Info_pos = df_holdout$Info_pos,
  Predicted_class = holdout_predictions_optimized$.pred_class
)

saveRDS(predictions_df, "mypreds.rds")
```

---

---

**========== End of Task II ==========**